American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Title : Text-Based Emotion Classification Using Naïve Bayes, Logistic Regression, and Support Vector Machine

Natural Language Processing

Mid-Term Project Report

Summer 2025-2026

Section: A

Group:11

Project Contributions

SL #	Student Name & ID	Contributions (mention every part of the project where you contributed)
1.	Name: Shohag Rana ID: 23-54897-3	Work with dataset -1
2.	Name: MD SHAHARIAR HOSSEN LABIB ID: 23-54489-3	Work with dataset-2
3.		
4.		

DataSet 1 Description :

It's a Emotion detection dataset .

Topic: Emotion Detection

Data set 1 Link: https://data.mendeley.com/datasets/dp338mw226/3

date :4 June 2026 (publish date)

Site : Mendeley

1. DATASET OVERVIEW

- File Name: Emotion_Sentiment_DataSet.csv
- Total Records: 160,000
- Features/Columns: Unnamed: 0 (Index), Text, Emotion
- Data Source: Collected from social media platforms and customized by combining/filtering two different existing datasets.

1. CONTENT & CLASS DISTRIBUTION

The dataset is labeled into 10 distinct classes representing standard emotions, general sentiment states, and behavioral/psychological indicators:

- love: 39,553 records
- happiness: 27,175 records
- sadness: 17,481 records
- Normal: 16,351 records
- hate: 15,267 records
- anger: 12,336 records
- Depression: 10,333 records
- fun: 10,075 records
- surprise: 6,954 records
- worry: 4,475 records

1. COLUMN DESCRIPTIONS

- Unnamed: 0: A unique identifier/index for each row.
- Text: Raw or semi-processed textual expressions collected from social media users.
- Emotion: The target categorical label assigned to the text (representing sentiment, emotion, or psychological indicator).

Project Implementation Detail

Task 1: Import Required Libraries

Description of the Task

The first step of this project is to import all the required Python libraries that will be used throughout the implementation. Each library provides specific functionalities for data handling, natural language processing (NLP), machine learning, and performance evaluation.

The imported libraries prepare the programming environment so that all subsequent tasks can be executed without errors. They allow the project to read the dataset, preprocess text, convert text into numerical features, train machine learning models, and evaluate the prediction results. Since different stages of the project require different tools, importing all necessary libraries at the beginning improves code organization and readability.

Code :

```
import pandas as pd
import numpy as np


import nltk
import string
import spacy
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import (
    accuracy_score,
```

```
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
print("done")
```

Sample Output

done

The output confirms that all required libraries have been imported successfully and the Python environment is ready for the remaining tasks.

Code Description

The code imports all the required libraries used throughout the project.

- **Pandas** and **NumPy** are imported for data handling and numerical operations.
- **NLTK** and **spaCy** provide NLP functionalities such as tokenization, stop word removal, stemming, and lemmatization.
- **Scikit-learn** modules are imported for data splitting, feature extraction using **Bag of Words** and **TF-IDF**, implementing the **Multinomial Naïve Bayes**, **Logistic Regression**, and **LinearSVC** algorithms, and evaluating model performance using different metrics.
- Finally, print("done") confirms that all libraries have been imported successfully.

Task 2: Download Required NLTK Resources

Description of the Task

Before using NLTK for text preprocessing, the required language resources must be downloaded. In this project, the **punkt** tokenizer and the **stopwords** corpus are downloaded. These resources enable NLTK to perform tokenization and stop word removal correctly. Downloading them ensures that the necessary data is available locally before the preprocessing tasks begin.

Code

```
nltk.download('punkt')

nltk.download('stopwords')
```

Sample Output

[nltk_data] Downloading package punkt...
[nltk_data] Package punkt is already up-to-date!

[nltk_data] Downloading package stopwords...
[nltk_data] Package stopwords is already up-to-date!

Code Description

- nltk.download('punkt') downloads the **Punkt tokenizer**, which is used for splitting text into words or sentences.
- nltk.download('stopwords') downloads the **English stop words** dataset used to remove common words during preprocessing.
- These resources only need to be downloaded once. If they are already installed, NLTK displays that they are up to date.

Task 3: Load the Dataset

Description of the Task

In this task, the emotion sentiment dataset is loaded into the program using the Pandas library. The dataset is stored in a CSV (Comma-Separated Values) file, which is one of the most common formats for storing tabular data. After loading the dataset, it is stored in a DataFrame named df, allowing the data to be easily accessed, analyzed, and processed in the following tasks.

Code

```
df = pd.read_csv("Emotion_Sentiment_DataSet.csv")
```

Sample Output

This code does not produce any direct output.

Code Description

- pd.read_csv() is used to read data from a CSV file.
- "Emotion_Sentiment_DataSet.csv" specifies the name of the dataset file.
- The loaded data is stored in the DataFrame df for further preprocessing, analysis, and model training.

Task 4: Explore and Analyze the Dataset

Description of the Task

Before applying any preprocessing or machine learning techniques, it is important to understand the structure and quality of the dataset. In this task, the dataset is explored by checking its dimensions, column names, missing values, and class distribution. An unnecessary column is then removed, and the first few records are displayed to verify that the dataset is ready for preprocessing.

Code :

```
df.shape


df.columns


df.isnull().sum()


df["Emotion"].value_counts()


df = df.drop(columns=["Unnamed: 0"])


df.head()
```

Sample Output:

```
(160000, 3)
```

```
Index(['Unnamed: 0', 'Text', 'Emotion'], dtype='str')
Unnamed: 0    0
Text          8
Emotion       0
dtype: int64
```

```
Emotion
love          39553
happiness     27175
sadness       17481
Normal        16351
hate          15267
anger         12336
Depression    10333
fun           10075
surprise       6954
worry          4475
Name: count, dtype: int64
```

	Text	Emotion	
0	i feel jealous becasue i wanted that kind of l...	love	
1	i feel like she has taken on the role of a gra...	love	
2	i feel like im back to the arms of a beloved l...	love	
3	i am feeling so festive right now and not just...	love	

4	i don t discuss even my feelings for beloved w...	Love

Code Description

- df.shape returns the number of rows and columns in the dataset.
- df.columns displays the names of all columns.
- df.isnull().sum() checks for missing values in each column.
- df["Emotion"].value_counts() shows the number of samples in each emotion class.
- df.drop(columns=["Unnamed: 0"]) removes the unnecessary index column.
- df.head() displays the first five rows to verify the cleaned dataset.

Task 5: Initialize the spaCy Language Model

Description of the Task

In this task, a blank English language model is created using the **spaCy** library. This model is used to perform **tokenization**, which splits the text into individual words or tokens. Since only tokenization is required at this stage, a blank model is sufficient and more efficient than loading the full English model.

Code

```
nlp = spacy.blank("en")
```

Sample Output

This code does not produce any visible output. It creates an English NLP pipeline and stores it in the variable nlp.

Code Description

- spacy.blank("en") creates a blank English language model.
- The model is stored in the nlp variable.
- It is used later to tokenize the text efficiently.

Task 6: Tokenization

Description of the Task

In this task, the text data is converted into individual **tokens** using the **spaCy** tokenizer. Before tokenization, any missing values are replaced with empty strings, and all text entries are converted to string format to ensure consistent processing. The generated tokens are then stored in a new column named **tokens**, which serves as the foundation for the subsequent preprocessing steps, such as case folding, stop word removal, and stemming.

Code

```
texts = df["Text"].fillna("").astype(str)

docs = nlp.pipe(texts, batch_size=512)

df["tokens"] = [[token.text for token in doc] for doc in docs]

df["tokens"].head(10)
```

Sample Output

```
0    [i, feel, jealous, becasue, i, wanted, that, k...
1    [i, feel, like, she, has, taken, on, the, role...
2    [i, feel, like, i, m, back, to, the, arms, of,...
3    [i, am, feeling, so, festive, right, now, and,...
4    [i, don, t, discuss, even, my, feelings, for, ...
5    [i, also, love, seeing, a, star, emerge, and, ...
6    [i, m, feeling, kinda, shaky, my, mind, is, fu...
7    [i, love, running, because, i, feel, strong, a...
8    [i, feel, that, popular, bloggers, do, nt, pos...
```

```
9    [i, can, never, fall, in, love, with, anyone, ...
Name: tokens, dtype: object
```

Code Description

- fillna("") replaces missing values with empty strings to prevent processing errors.
- astype(str) converts all entries into string format.
- nlp.pipe() tokenizes all text efficiently by processing multiple documents in batches.
- batch_size=512 improves processing speed by handling 512 texts at a time.
- The tokenized words are extracted using token.text and stored in the **tokens** column.
- df["tokens"].head(10) displays the first 10 tokenized records for verification

Task 7: Case Folding

Description of the Task

In this task, **case folding** is applied to the tokenized text. Case folding converts all uppercase letters into lowercase letters, ensuring that words with different letter cases are treated as the same word. For example, **"Happy"**, **"HAPPY"**, and **"happy"** are all converted to **"happy"**. This reduces duplicate words in the vocabulary and improves the consistency of the text data before feature extraction.

Code

```
df["tokens"] = df["tokens"].apply(
    lambda x: [word.lower() for word in x]
)
```

Sample Output

Before Case Folding

['I', 'Feel', 'Happy', 'TODAY', '!']

After Case Folding
['i', 'feel', 'happy', 'today', '!']

Code Description

- apply() applies the function to every row in the **tokens** column.
- word.lower() converts each token into lowercase.
- The updated lowercase tokens replace the original tokens in the **tokens** column.

Task 8: Stop Word Removal

Description of the Task

In this task, **stop word removal** is performed to eliminate common English words that usually do not contribute significant meaning to the text. Words such as **"the"**, **"is"**, **"and"**, and **"of"** appear very frequently but provide little information for emotion classification. Removing these words reduces noise, decreases the size of the vocabulary, and allows the machine learning models to focus on more meaningful words.

Code

```
stop_words = set(stopwords.words("english"))

df["tokens"] = df["tokens"].apply(
    lambda x: [word for word in x if word not in stop_words]
)
```

Sample Output

```
Text Emotion  \
0  i feel jealous becasue i wanted that kind of l...    love
1  i feel like she has taken on the role of a gra...    love
2  i feel like im back to the arms of a beloved l...    love
3  i am feeling so festive right now and not just...    love
4  i don t discuss even my feelings for beloved w...    love

                                              tokens
0  [feel, jealous, becasue, wanted, kind, love, t...
1  [feel, like, taken, role, grandmother, since, ...
2  [feel, like, back, arms, beloved, last, seen, ...
3  [feeling, festive, right, lovely, wintry, scen...
4         [discuss, even, feelings, beloved, anyone]
```

Code Description

- stopwords.words("english") loads the predefined list of English stop words from the NLTK library.
- set() stores the stop words as a set, making the search process faster.
- apply() processes each list of tokens in the **tokens** column.
- The list comprehension removes any token that exists in the stop words list.
- The filtered tokens are stored back in the **tokens** column.

Task 9: Stemming

Description of the Task

In this task, **stemming** is applied to the tokenized text using the **Porter Stemmer** provided by the NLTK library. Stemming reduces words to their root or base form by removing prefixes and suffixes. This helps group different forms of the same word into a single representation, reducing the vocabulary size and improving the efficiency of the machine learning models.

Code

```
stemmer = PorterStemmer()

df["tokens"] = df["tokens"].apply(
    lambda x: [stemmer.stem(word) for word in x]
)
```

Sample Output

```
Text Emotion  \
0  i feel jealous becasue i wanted that kind of l...    love
1  i feel like she has taken on the role of a gra...    love
2  i feel like im back to the arms of a beloved l...    love
3  i am feeling so festive right now and not just...    love
4  i don t discuss even my feelings for beloved w...    love

                                        tokens
0  [feel, jealou, becasu, want, kind, love, true,...
1  [feel, like, taken, role, grandmoth, sinc, bel...
2  [feel, like, back, arm, belov, last, seen, lon...
3  [feel, festiv, right, love, wintri, scene, wal...
4                [discuss, even, feel, belov, anyon]
```

Code Description

- PorterStemmer() creates a stemming object.
- apply() applies the stemming process to every list of tokens.
- stemmer.stem(word) converts each word into its stem (root) form.
- The stemmed words replace the original tokens in the **tokens** column.

Task 10: Lemmatization

Description of the Task

In this task, **lemmatization** is performed using the **spaCy** English language model. Unlike stemming, lemmatization converts words to their actual dictionary (base) form while considering the word's context and grammatical meaning. This produces more meaningful

and linguistically correct words, which helps improve the quality of the text before feature extraction and model training.

Code

```
nlp = spacy.load("en_core_web_sm")

df["lemmas"] = df["Text"].fillna("").apply(
   lambda text: [token.lemma_ for token in nlp(str(text))]
)
```

Sample Output

```
Text Emotion  \
0  i feel jealous becasue i wanted that kind of l...    love
1  i feel like she has taken on the role of a gra...    love
2  i feel like im back to the arms of a beloved l...    love
3  i am feeling so festive right now and not just...    love
4  i don t discuss even my feelings for beloved w...    love

                                        tokens  \
0  [feel, jealou, becasu, want, kind, love, true,...
1  [feel, like, taken, role, grandmoth, sinc, bel...
2  [feel, like, back, arm, belov, last, seen, lon...
3  [feel, festiv, right, love, wintri, scene, wal...
4                [discuss, even, feel, belov, anyon]

                                        lemmas
0  [I, feel, jealous, becasue, I, want, that, kin...
1  [I, feel, like, she, have, take, on, the, role...
2  [I, feel, like, I, m, back, to, the, arm, of, ...
3  [I, be, feel, so, festive, right, now, and, no...
4  [I, don, t, discuss, even, my, feeling, for, b...
```

Code Description

- spacy.load("en_core_web_sm") loads the pre-trained English language model.
- fillna("") replaces missing values with empty strings.
- apply() processes each text in the dataset.
- token.lemma_ extracts the lemma (base form) of each word.

- The lemmatized words are stored in a new column named **lemmas**.

Task 11: Create Processed Text

Description of the Task

After completing the lemmatization process, the generated lemmas are combined into complete sentences to create the final processed text. Since the machine learning models and vectorizers require text in string format rather than lists of words, the lemmas are joined into a single sentence for each record. The processed dataset is then saved as a pickle file, and the first few rows are displayed to verify the preprocessing results.

Code

```
df["processed_text"] = df["lemmas"].apply(" ".join)

df.to_pickle("step6_process_text.pkl")

df.head()
```

Sample Output

Text	Emotion	tokens	lemmas	processed_text
i feel jealous...	love	[feel, jealou, ...]	[I, feel, jealous, ...]	I feel jealous becasue I want that kind of love...
i feel like...	love	[feel, like, ...]	[I, feel, like, ...]	I feel like she have take on the role...

Code Description

- apply(" ".join) joins all lemmas into a single text string.
- The combined text is stored in the **processed_text** column.

- to_pickle() saves the processed dataset for future use.
- head() displays the first five rows to verify that the new column has been created successfully.

Task 12: Split the Dataset and Extract Bag of Words (BoW) Features

Description of the Task

In this task, the processed text and corresponding emotion labels are separated into **features** and **target values**. The dataset is then divided into **training** and **testing** sets using an **80:20 ratio**. The training data is used to train the machine learning models, while the testing data is used to evaluate their performance on unseen data. After splitting the dataset, the **Bag of Words (BoW)** technique is applied to convert the text into numerical feature vectors that can be processed by machine learning algorithms.

Code

```
X = df["processed_text"]
y = df["Emotion"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

bow = CountVectorizer()

X_train_bow = bow.fit_transform(X_train)
X_test_bow = bow.transform(X_test)
```

Sample Output

This code does not produce direct output. It creates the variables

Code Description

- X stores the processed text, while y stores the corresponding emotion labels.
- train_test_split() divides the dataset into **80% training** and **20% testing** data.
- random_state=42 ensures the data is split the same way every time the code is executed.
- CountVectorizer() creates the Bag of Words model.
- fit_transform() learns the vocabulary from the training data and converts it into numerical vectors.
- transform() converts the testing data using the same vocabulary learned from the training data.

Task 13: Train and Evaluate Multinomial Naïve Bayes using Bag of Words (BoW)

Description of the Task

In this task, the **Multinomial Naïve Bayes (MultinomialNB)** classifier is trained using the **Bag of Words (BoW)** feature representation. After training, the model predicts the emotion labels of the testing dataset. The model's performance is then evaluated using several metrics, including **Accuracy, Precision, Recall, F1-score, Confusion Matrix,** and **Classification Report**. These metrics help determine how effectively the model classifies different emotions.

Code

```
nb = MultinomialNB()

nb.fit(X_train_bow, y_train)

y_pred = nb.predict(X_test_bow)

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred, average="weighted"))
```

```
print("F1 Score :", f1_score(y_test, y_pred, average="weighted"))

print(confusion_matrix(y_test, y_pred))

print(classification_report(y_test, y_pred))
```

Sample Output

Accuracy : 0.86146875
Precision: 0.8908691434209736
Recall : 0.86146875
F1 Score : 0.8600100409535867

[[2000 11 1 0 7 12 12 11 1 1]

[607 1809 19 23 148 46 581 55 7 6]

[91 5 2101 0 68 17 179 60 2 0]

[115 4 5 1567 19 5 301 28 2 0]

[104 3 3 11 5037 12 200 27 0 0]

[109 3 3 4 9 2675 206 18 0 0]

[114 6 7 3 30 13 7529 29 2 0]

[244 3 1 4 23 4 33 3269 0 0]

[85 0 3 4 19 11 56 80 1143 0]

[200 7 8 3 62 6 122 89 1 437]]

	precision	recall	f1-score	support
Depression	0.55	0.97	0.70	2056
Normal	0.98	0.55	0.70	3301
anger	0.98	0.83	0.90	2523
fun	0.97	0.77	0.86	2046
happiness	0.93	0.93	0.93	5397
hate	0.96	0.88	0.92	3027

love	*0.82*	*0.97*	*0.89*	*7733*
sadness	*0.89*	*0.91*	*0.90*	*3581*
surprise	*0.99*	*0.82*	*0.89*	*1401*
worry	*0.98*	*0.47*	*0.63*	*935*
accuracy			*0.86*	*32000*
macro avg	*0.90*	*0.81*	*0.83*	*32000*
weighted avg	*0.89*	*0.86*	*0.86*	*32000*

Code Description

- MultinomialNB() creates the Naïve Bayes classification model.
- fit() trains the model using the **BoW training features** and their corresponding emotion labels.
- predict() predicts the emotion labels for the testing dataset.
- accuracy_score(), precision_score(), recall_score(), and f1_score() evaluate the model's overall performance.
- confusion_matrix() shows the number of correct and incorrect predictions for each emotion class.
- classification_report() provides a detailed summary of Precision, Recall, F1-score, and Support for every class.

Result Discussion

The **Multinomial Naïve Bayes** model achieved an **accuracy of 86.15%** using the **Bag of Words** representation. The weighted **Precision**, **Recall**, and **F1-score** are also close to **86–89%**, indicating that the model performs well overall. From the classification report, emotions such as **happiness**, **love**, **hate**, and **sadness** are classified with high accuracy, while **Depression** and **worry** are comparatively more difficult to classify because they share similar vocabulary with other emotion classes.

Task 14: Train and Evaluate Logistic Regression using Bag of Words (BoW)

Description of the Task

In this task, the **Logistic Regression** classifier is trained using the **Bag of Words (BoW)** feature representation. The model learns the relationship between the numerical text features and their corresponding emotion labels from the training dataset. After training, it predicts the emotions of the testing dataset. The model's performance is evaluated using **Accuracy, Precision, Recall, F1-score, Confusion Matrix,** and **Classification Report** to measure its classification effectiveness.

Code

```
lr = LogisticRegression(max_iter=1000)

lr.fit(X_train_bow, y_train)

y_pred_lr_bow = lr.predict(X_test_bow)

print("========== Logistic Regression (BoW) ==========")

print("Accuracy :", accuracy_score(y_test, y_pred_lr_bow))
print("Precision:", precision_score(y_test, y_pred_lr_bow, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_lr_bow, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_lr_bow, average="weighted"))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred_lr_bow))

print("\nClassification Report")
print(classification_report(y_test, y_pred_lr_bow))
```

Sample Output

```
========== Logistic Regression (BoW) ==========

Accuracy : 0.98390625
Precision: 0.9837587683364096
```

Recall : 0.98390625
F1 Score : 0.9837834960838183

========== Logistic Regression (BoW) ==========

Accuracy : 0.98390625

Precision: 0.9837587683364096

Recall : 0.98390625

F1 Score : 0.9837834960838183

Confusion Matrix

[[1858 119 5 5 12 17 10 26 0 4]

[61 3135 9 9 25 10 29 11 3 9]

[0 0 2518 0 2 2 1 0 0 0]

[2 6 1 2032 3 1 1 0 0 0]

[4 3 1 0 5371 10 8 0 0 0]

[7 1 2 0 6 3004 6 0 0 1]

[5 5 0 0 17 5 7701 0 0 0]

[14 0 0 3 8 2 12 3542 0 0]

[0 0 0 1 0 0 0 0 1400 0]

[2 0 2 1 3 0 3 0 0 924]]

Classification Report

	precision	recall	f1-score	support
Depression	0.95	0.90	0.93	2056
Normal	0.96	0.95	0.95	3301
anger	0.99	1.00	1.00	2523
fun	0.99	0.99	0.99	2046
happiness	0.99	1.00	0.99	5397
hate	0.98	0.99	0.99	3027
love	0.99	1.00	0.99	7733

```
   sadness      0.99     0.99     0.99     3581
  surprise      1.00     1.00     1.00     1401
     worry      0.99     0.99     0.99      935

  accuracy                          0.98    32000
 macro avg      0.98     0.98     0.98    32000
weighted avg    0.98     0.98     0.98    32000
```

Code Description

- LogisticRegression(max_iter=1000) creates the Logistic Regression classifier and sets the maximum number of training iterations to **1000** to ensure the model converges.
- fit() trains the model using the **BoW training features** and their corresponding emotion labels.
- predict() predicts the emotion labels for the testing dataset.
- The evaluation metrics (**Accuracy, Precision, Recall, and F1-score**) measure the overall performance of the model.
- confusion_matrix() and classification_report() provide a detailed analysis of the model's predictions.

Result Discussion

The **Logistic Regression** model achieved an **accuracy of 98.39%** using the **Bag of Words** representation. It also obtained **98.38% Precision**, **98.39% Recall**, and **98.38% F1-score**, indicating excellent classification performance. The classification report shows that most emotion classes have **Precision**, **Recall**, and **F1-score** close to **1.00**, demonstrating that the model correctly classified the majority of the testing samples. Compared to the **Multinomial Naïve Bayes** model, Logistic Regression produced significantly better results and fewer misclassifications.

Task 15: Train and Evaluate Linear Support Vector Classifier (LinearSVC) using Bag of Words (BoW)

Description of the Task

In this task, the **Linear Support Vector Classifier (LinearSVC)** is trained using the **Bag of Words (BoW)** feature representation. The model learns to classify emotions by finding the optimal decision boundary that best separates different emotion classes. After training, the model predicts the emotion labels of the testing dataset. The performance of the classifier is then evaluated using **Accuracy, Precision, Recall, F1-score, Confusion Matrix,** and **Classification Report**.

Code

```
svm = LinearSVC()


svm.fit(X_train_bow, y_train)

y_pred = svm.predict(X_test_bow)

print("========== Linear SVC (BoW) ==========")

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred, average="weighted"))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report")
print(classification_report(y_test, y_pred))
```

Sample Output

```
========== Linear SVC (BoW) ==========

Accuracy : 0.982125
Precision: 0.9819106505207507
Recall   : 0.982125
F1 Score : 0.9818802229047702


========== Linear SVC (BoW) ==========

Accuracy : 0.982125

Precision: 0.9819106505207507

Recall   : 0.982125

F1 Score : 0.9818802229047702


Confusion Matrix

[[1848   96    5    9   11   23   14   37    5    8]

[  78 3027   16   20   34   29   48   26    9   14]

[   0    0 2523    0    0    0    0    0    0    0]

[   0    2    0 2041    3    0    0    0    0    0]

[   4    2    0    0 5377    8    4    2    0    0]

[   0    0    2    2    1 3017    4    0    0    1]

[   3    2    0    2    7    7 7710    0    0    2]

[  14    0    0    0    4    0    8 3555    0    0]

[   0    0    0    0    0    0    0    0 1401    0]

[   2    0    2    0    0    0    2    0    0  929]]


Classification Report

              precision    recall  f1-score   support


  Depression       0.95      0.90      0.92      2056

      Normal       0.97      0.92      0.94      3301

       anger       0.99      1.00      1.00      2523
```

```
              precision    recall  f1-score   support

         fun       0.98      1.00      0.99      2046
   happiness       0.99      1.00      0.99      5397
        hate       0.98      1.00      0.99      3027
        love       0.99      1.00      0.99      7733
     sadness       0.98      0.99      0.99      3581
    surprise       0.99      1.00      1.00      1401
       worry       0.97      0.99      0.98       935

    accuracy                           0.98     32000
   macro avg       0.98      0.98      0.98     32000
weighted avg       0.98      0.98      0.98     32000
```

Code Description

- LinearSVC() creates the Linear Support Vector Classifier model.
- fit() trains the model using the **BoW training data** and their corresponding emotion labels.
- predict() predicts the emotion labels for the testing dataset.
- The model is evaluated using **Accuracy, Precision, Recall, and F1-score**.
- confusion_matrix() and classification_report() provide detailed information about the model's classification performance.

Result Discussion

The **LinearSVC** model achieved an **accuracy of 98.21%** using the **Bag of Words** representation. It also obtained **98.19% Precision**, **98.21% Recall**, and **98.19% F1-score**, demonstrating excellent classification performance. The classification report shows that most emotion classes achieved very high scores, with several classes obtaining **1.00** recall or precision. Compared with **Multinomial Naïve Bayes**, LinearSVC performed significantly better. Its performance is also very close to **Logistic Regression**, although **Logistic Regression achieved a slightly higher accuracy** on this dataset.

Task 16: Extract TF-IDF Features

Description of the Task

In this task, the **Term Frequency–Inverse Document Frequency (TF-IDF)** technique is used to convert the processed text into numerical feature vectors. Unlike **Bag of Words (BoW)**, which only counts the frequency of words, **TF-IDF** assigns higher weights to important words and lower weights to very common words. These numerical vectors are then used as input for the machine learning models.

Code

```
tfidf = TfidfVectorizer()

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

Sample Output

This code does not produce any direct output. It creates two TF-IDF feature matrices.

Code Description

- TfidfVectorizer() creates the TF-IDF vectorizer.
- fit_transform() learns the vocabulary from the training data and converts it into TF-IDF feature vectors.
- transform() converts the testing data into TF-IDF vectors using the same vocabulary learned from the training data.

Task 17: Train and Evaluate Multinomial Naïve Bayes using TF-IDF

Description of the Task

In this task, the **Multinomial Naïve Bayes (MultinomialNB)** classifier is trained using the **TF-IDF** feature representation. The model learns from the TF-IDF vectors of the training data and predicts the emotion labels for the testing data. The performance of the model is evaluated using **Accuracy, Precision, Recall, F1-score, Confusion Matrix,** and **Classification Report**.

Code

```
nb_tfidf = MultinomialNB()

nb_tfidf.fit(X_train_tfidf, y_train)

y_pred_nb_tfidf = nb_tfidf.predict(X_test_tfidf)

print("Accuracy :", accuracy_score(y_test, y_pred_nb_tfidf))
print("Precision:", precision_score(y_test, y_pred_nb_tfidf, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_nb_tfidf, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_nb_tfidf, average="weighted"))

print("\nConfusion Matrix")

print(confusion_matrix(y_test, y_pred_nb_tfidf))



print("\nClassification Report")
print(classification_report(y_test, y_pred_nb_tfidf))
```

Sample Output

```
Accuracy : 0.60365625
Precision: 0.7933429718002504
Recall   : 0.60365625
F1 Score : 0.5772571927518781
```

```
Accuracy : 0.60365625
Precision: 0.7933429718002504
Recall   : 0.60365625
F1 Score : 0.5772571927518781

Confusion Matrix
[[1536   6   1   0  24   4 474  11   0   0]
 [ 121 1064   4   2 210  14 1857  29   0   0]
 [  6   0 799   0 204   4 1402 108   0   0]
 [ 14   5   2 278  54   1 1676  16   0   0]
 [  1   2   0   1 4039   0 1350   4   0   0]
 [ 10   3   2   0  26 1072 1908   6   0   0]
 [  2   3   0   0  29   5 7692   2   0   0]
 [ 85   3   1   2  87   2 822 2579   0   0]
 [  6   9   2   1 125   2 882 133 241   0]
 [ 50   1   2   0 167   3 639  56   0  17]]

Classification Report
              precision    recall  f1-score   support

  Depression       0.84      0.75      0.79      2056
      Normal       0.97      0.32      0.48      3301
       anger       0.98      0.32      0.48      2523
         fun       0.98      0.14      0.24      2046
   happiness       0.81      0.75      0.78      5397
        hate       0.97      0.35      0.52      3027
        love       0.41      0.99      0.58      7733
     sadness       0.88      0.72      0.79      3581
    surprise       1.00      0.17      0.29      1401
       worry       1.00      0.02      0.04       935

    accuracy                           0.60     32000
   macro avg       0.88      0.45      0.50     32000
weighted avg       0.79      0.60      0.58     32000
```

Code Description

- MultinomialNB() creates the Naïve Bayes classifier.
- fit() trains the model using the **TF-IDF training features**.
- predict() predicts the emotion labels for the testing data.
- The model is evaluated using **Accuracy, Precision, Recall, F1-score, Confusion Matrix,** and **Classification Report**.

Result Discussion

The **Multinomial Naïve Bayes** model achieved an **accuracy of 60.37%** using the **TF-IDF** representation. The weighted **Precision** is **79.33%**, **Recall** is **60.37%**, and **F1-score** is **57.73%**. Compared to the **Bag of Words** representation (**86.15% accuracy**), the TF-IDF representation produced significantly lower performance for this model. This indicates that, for this dataset, **Multinomial Naïve Bayes performs better with Bag of Words than with TF-IDF**.

Task 18: Train and Evaluate Logistic Regression using TF-IDF

Description of the Task

In this task, the **Logistic Regression** classifier is trained using the **TF-IDF** feature representation. The model learns the relationship between the TF-IDF features and the corresponding emotion labels from the training dataset. After training, it predicts the emotion labels for the testing dataset. The performance of the model is evaluated using **Accuracy, Precision, Recall, F1-score, Confusion Matrix,** and **Classification Report**.

Code

```
lr_tfidf = LogisticRegression(max_iter=1000)

lr_tfidf.fit(X_train_tfidf, y_train)

y_pred_lr_tfidf = lr_tfidf.predict(X_test_tfidf)

print("Accuracy :", accuracy_score(y_test, y_pred_lr_tfidf))
print("Precision:", precision_score(y_test, y_pred_lr_tfidf, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_lr_tfidf, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_lr_tfidf, average="weighted"))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred_lr_tfidf))
```

```
print("\nClassification Report")
print(classification_report(y_test, y_pred_lr_tfidf))
```

Sample Output

```
Accuracy : 0.9710625

Precision: 0.9708740063897358

Recall   : 0.9710625

F1 Score : 0.9708836354892991


Confusion Matrix

[[1839  119    0    2   20   22   17   32    0    5]

[  87 3015   12   18   43   17   67   24    5   13]

[   3    7 2505    0    1    2    4    0    0    1]

[   5   15    9 1992   10    4   10    0    0    1]

[   7   14    3    0 5340    6   25    0    0    2]

[  10    6    8    0   10 2976   16    0    0    1]

[   4    8    3    1   29   18 7668    0    0    2]

[  19   10    1    7   25   11   32 3472    2    2]

[   1    8    4    2    0    4    9    0 1373    0]

[   5    1    4    4    7    2   12    0    6  894]]


Classification Report

              precision    recall  f1-score   support


  Depression       0.93      0.89      0.91      2056

      Normal       0.94      0.91      0.93      3301

       anger       0.98      0.99      0.99      2523

         fun       0.98      0.97      0.98      2046

   happiness       0.97      0.99      0.98      5397
```

```
      hate     0.97     0.98     0.98     3027
      love     0.98     0.99     0.98     7733
   sadness     0.98     0.97     0.98     3581
  surprise     0.99     0.98     0.99     1401
     worry     0.97     0.96     0.96      935

  accuracy                          0.97    32000
 macro avg     0.97     0.96     0.97    32000
weighted avg     0.97     0.97     0.97    32000
```

Code Description

- LogisticRegression(max_iter=1000) creates the Logistic Regression model with a maximum of **1000 iterations**.
- fit() trains the model using the **TF-IDF training features**.
- predict() predicts the emotion labels for the testing data.
- The evaluation metrics measure the model's performance.
- confusion_matrix() and classification_report() provide detailed classification results.

Result Discussion

The **Logistic Regression** model achieved an **accuracy of 97.11%** using the **TF-IDF** representation. The weighted **Precision**, **Recall**, and **F1-score** are all approximately **97%**, indicating excellent classification performance. Compared to **Logistic Regression with Bag of Words (98.39%)**, the TF-IDF model achieved slightly lower accuracy. However, it still performed much better than **Multinomial Naïve Bayes with TF-IDF**, demonstrating that Logistic Regression works effectively with TF-IDF features.

Task 19: Train and Evaluate Linear Support Vector Classifier (LinearSVC) using TF-IDF

Description of the Task

In this task, the **Linear Support Vector Classifier (LinearSVC)** is trained using the **TF-IDF** feature representation. The model learns to classify different emotions by finding the optimal decision boundary that best separates the emotion classes. After training, the model predicts the emotion labels of the testing dataset. The model's performance is evaluated using **Accuracy, Precision, Recall, F1-score, Confusion Matrix,** and **Classification Report**.

Code

```
svm_tfidf = LinearSVC()

svm_tfidf.fit(X_train_tfidf, y_train)

y_pred_svm_tfidf = svm_tfidf.predict(X_test_tfidf)

print("Accuracy :", accuracy_score(y_test, y_pred_svm_tfidf))
print("Precision:", precision_score(y_test, y_pred_svm_tfidf, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_svm_tfidf, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_svm_tfidf, average="weighted"))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred_svm_tfidf))

print("\nClassification Report")
print(classification_report(y_test, y_pred_svm_tfidf))
```

Sample Output

```
Accuracy : 0.980375

Precision: 0.9801996025361107

Recall   : 0.980375

F1 Score : 0.9801319850919021
```

Confusion Matrix

```
[[1870   90    2    3   16   19   11   37    1    7]
 [  73 3005   15   20   53   23   63   27    9   13]
 [   0    0 2520    0    0    2    1    0    0    0]
 [   0    0    0 2040    1    1    4    0    0    0]
 [   5    3    1    0 5368    8   10    0    0    2]
 [   0    1    8    0    8 3003    6    0    0    1]
 [   2    2    7    0   17    7 7694    0    1    3]
 [   7    0    0    2    5    3   13 3551    0    0]
 [   0    2    0    0    0    0    1    0 1398    0]
 [   0    0    2    1    3    0    5    0    1  923]]
```

Classification Report

	precision	recall	f1-score	support
Depression	0.96	0.91	0.93	2056
Normal	0.97	0.91	0.94	3301
anger	0.99	1.00	0.99	2523
fun	0.99	1.00	0.99	2046
happiness	0.98	0.99	0.99	5397
hate	0.98	0.99	0.99	3027
love	0.99	0.99	0.99	7733
sadness	0.98	0.99	0.99	3581
surprise	0.99	1.00	0.99	1401
worry	0.97	0.99	0.98	935
accuracy			0.98	32000
macro avg	0.98	0.98	0.98	32000
weighted avg	0.98	0.98	0.98	32000

Code Description

- LinearSVC() creates the Linear Support Vector Classifier model.
- fit() trains the model using the **TF-IDF training features** and emotion labels.
- predict() predicts the emotion labels for the testing data.
- The model is evaluated using **Accuracy, Precision, Recall, and F1-score**.
- confusion_matrix() and classification_report() provide detailed information about the model's performance for each emotion class.

Result Discussion

The **LinearSVC** model achieved an **accuracy of 98.04%** using the **TF-IDF** representation. It also obtained **98.02% Precision**, **98.04% Recall**, and **98.01% F1-score**, showing excellent classification performance. Compared with **LinearSVC using Bag of Words (98.21%)**, the TF-IDF representation produced a slightly lower accuracy. However, the difference is very small, indicating that **LinearSVC performs very well with both feature representations**.

Task 20: Compare Predictions from All Trained Models

Description of the Task

In this task, a custom function named **show_all_model_prediction()** is created to test all six trained models on a single input sentence. The function converts the input text into the appropriate feature representation (**BoW** or **TF-IDF**), predicts the emotion using each model, calculates the confidence score for every emotion class, and displays the results in a readable table. This allows us to compare how different algorithms classify the same text.

Code

```
import numpy as np
import pandas as pd



def show_all_model_prediction(text):

models = {
```

```
"Naive Bayes (BOW)": (nb, bow),
"Logistic Regression (BOW)": (lr, bow),
"SVM (BOW)": (svm, bow),

"Naive Bayes (TF-IDF)": (nb_tfidf, tfidf),
"Logistic Regression (TF-IDF)": (lr_tfidf, tfidf),
"SVM (TF-IDF)": (svm_tfidf, tfidf)
}


print("="*70)
print("Input Text:")
print(text)
print("="*70)


for name, (model, vectorizer) in models.items():

try:

x = vectorizer.transform([text])

prediction = model.predict(x)[0]


if hasattr(model, "predict_proba"):

probabilities = model.predict_proba(x)[0]


else:

scores = model.decision_function(x)

if len(scores.shape) > 1:
scores = scores[0]


exp_scores = np.exp(
scores - np.max(scores)
)

probabilities = exp_scores / exp_scores.sum()
```

```
classes = model.classes_


result = pd.DataFrame({
"Class": classes,
"Confidence": probabilities * 100
})


result = result.sort_values(
by="Confidence",
ascending=False
)


print("\nMODEL:", name)
print("Prediction:", prediction)

print(
result.to_string(
index=False,
formatters={
"Confidence": "{:.2f}%".format
}
)
)

print("-"*70)


except Exception as e:

print("\nMODEL:", name)
print("ERROR:", e)
print("-"*70)
show_all_model_prediction(
"""

can't do anything in life
"""
)
```

Sample Output

Input Text

can't do anything in life

```
======================================================================
Input Text:



can't do anything in life

======================================================================


MODEL: Naive Bayes (BOW)
Prediction: Depression
    Class Confidence
Depression    64.32%
happiness    13.25%
     love    10.32%
     hate     3.34%
  sadness     3.27%
   Normal     2.30%
      fun     1.55%
    anger     1.26%
 surprise     0.23%
    worry     0.17%
----------------------------------------------------------------------


MODEL: Logistic Regression (BOW)
Prediction: Normal
    Class Confidence
```

```
   Normal    96.46%
Depression     2.76%
happiness     0.21%
    love     0.17%
   anger     0.13%
  sadness     0.09%
    hate     0.09%
     fun     0.07%
   worry     0.01%
 surprise     0.01%
---------------------------------------------------------------------

MODEL: SVM (BOW)
Prediction: Normal
    Class Confidence
   Normal    57.32%
Depression     9.83%
   anger     6.02%
happiness     4.62%
 surprise     4.26%
   worry     4.03%
  sadness     4.02%
     fun     3.96%
    love     3.59%
    hate     2.35%
---------------------------------------------------------------------

MODEL: Naive Bayes (TF-IDF)
Prediction: love
    Class Confidence
    love    28.41%
```

```
happiness    24.51%
Depression    24.16%
  sadness     7.10%
     hate     6.17%
   Normal     3.33%
    anger     2.73%
      fun     2.58%
 surprise     0.68%
    worry     0.32%
----------------------------------------------------------------------

MODEL: Logistic Regression (TF-IDF)
Prediction: Depression
    Class Confidence
Depression    78.07%
   Normal    19.04%
happiness     0.68%
     love     0.65%
      fun     0.52%
  sadness     0.37%
    anger     0.31%
     hate     0.23%
    worry     0.08%
 surprise     0.06%
----------------------------------------------------------------------

MODEL: SVM (TF-IDF)
Prediction: Depression
    Class Confidence
Depression    32.51%
   Normal    27.35%
```

```
     fun    7.19%
    love    5.94%
   anger    5.79%
 sadness    4.81%
happiness    4.48%
 surprise    4.24%
   worry    4.23%
    hate    3.46%
--------------------------------------------------------------------
```

Code Description

- A dictionary named models stores all six trained models along with their corresponding vectorizers.
- The input text is converted into **BoW** or **TF-IDF** features using transform().
- predict() determines the predicted emotion for each model.
- For models that support predict_proba() (Naïve Bayes and Logistic Regression), probability values are obtained directly.
- Since **LinearSVC** does not support predict_proba(), decision_function() is used and the scores are converted into approximate confidence values using the **Softmax** formula.
- The confidence scores are displayed in descending order using a Pandas DataFrame.

Result Discussion

The function successfully compares the predictions of all six trained models on the same input sentence. For the text **"can't do anything in life"**, different models predicted different

emotions. Most models (**Logistic Regression with TF-IDF** and **LinearSVC with TF-IDF**, and Naïve Bayes with BoW) predicted **Depression**, while the **BoW Logistic Regression** and **BoW LinearSVC** predicted **Normal**. This comparison shows that different algorithms and feature representations may produce different predictions for the same text because each model learns patterns differently.

PROJECT CODE

```
import pandas as pd
import numpy as np

import nltk
import string
import spacy
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

print('done')

nltk.download('punkt')
nltk.download('stopwords')

df = pd.read_csv("Emotion_Sentiment_DataSet.csv")

df.head(10)
```

```
df.shape

df.columns

df.isnull().sum()

df["Emotion"].value_counts()

df = df.drop(columns=["Unnamed: 0"])

df.head()

nlp = spacy.blank("en")

texts = df["Text"].fillna("").astype(str)

docs = nlp.pipe(texts, batch_size=512)

df["tokens"] = [[token.text for token in doc] for doc in docs]

df.to_pickle("df_tokenized.pkl")

df["tokens"].head(10)

df["tokens"] = df["tokens"].apply(lambda x: [word.lower() for word in x])

df.to_pickle("step2_case_folding.pkl")

stop_words = set(stopwords.words("english"))

df["tokens"] = df["tokens"].apply(
    lambda x: [word for word in x if word not in stop_words]
)

df.to_pickle("step3_stop_word_removed.pkl")

print(pd.read_pickle('step3_stop_word_removed.pkl').head())

stemmer = PorterStemmer()

df["tokens"] = df["tokens"].apply(
    lambda x: [stemmer.stem(word) for word in x]
)
```

```
df.to_pickle("step4_stemmer_removed.pkl")

print(pd.read_pickle('step4_stemmer_removed.pkl').head())

nlp = spacy.load("en_core_web_sm")

df["lemmas"] = df["Text"].fillna("").apply(
    lambda text: [token.lemma_ for token in nlp(str(text))]
)

df.to_pickle("step5_lemma.pkl")

print(pd.read_pickle('step5_lemma.pkl').head())

df["processed_text"] = df["lemmas"].apply(" ".join)

df.to_pickle("step6_process_text.pkl")

df.head()

X = df["processed_text"]
y = df["Emotion"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

bow = CountVectorizer()

X_train_bow = bow.fit_transform(X_train)
X_test_bow = bow.transform(X_test)

nb = MultinomialNB()

nb.fit(X_train_bow, y_train)

y_pred = nb.predict(X_test_bow)

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred, average="weighted"))
```

```
print(confusion_matrix(y_test, y_pred))

print(classification_report(y_test, y_pred))

lr = LogisticRegression(max_iter=1000)

lr.fit(X_train_bow, y_train)

y_pred_lr_bow = lr.predict(X_test_bow)

print("========== Logistic Regression (BoW) ==========")

print("Accuracy :", accuracy_score(y_test, y_pred_lr_bow))
print("Precision:", precision_score(y_test, y_pred_lr_bow, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_lr_bow, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_lr_bow, average="weighted"))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred_lr_bow))

print("\nClassification Report")
print(classification_report(y_test, y_pred_lr_bow))

svm = LinearSVC()

svm.fit(X_train_bow, y_train)

y_pred = svm.predict(X_test_bow)

print("========== Linear SVC (BoW) ==========")

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred, average="weighted"))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report")
print(classification_report(y_test, y_pred))

tfidf = TfidfVectorizer()

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

```
nb_tfidf = MultinomialNB()

nb_tfidf.fit(X_train_tfidf, y_train)

y_pred_nb_tfidf = nb_tfidf.predict(X_test_tfidf)

print("Accuracy :", accuracy_score(y_test, y_pred_nb_tfidf))
print("Precision:", precision_score(y_test, y_pred_nb_tfidf, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_nb_tfidf, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_nb_tfidf, average="weighted"))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred_nb_tfidf))

print("\nClassification Report")
print(classification_report(y_test, y_pred_nb_tfidf))

lr_tfidf = LogisticRegression(max_iter=1000)

lr_tfidf.fit(X_train_tfidf, y_train)

y_pred_lr_tfidf = lr_tfidf.predict(X_test_tfidf)

print("Accuracy :", accuracy_score(y_test, y_pred_lr_tfidf))
print("Precision:", precision_score(y_test, y_pred_lr_tfidf, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_lr_tfidf, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_lr_tfidf, average="weighted"))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred_lr_tfidf))

print("\nClassification Report")
print(classification_report(y_test, y_pred_lr_tfidf))

svm_tfidf = LinearSVC()

svm_tfidf.fit(X_train_tfidf, y_train)

y_pred_svm_tfidf = svm_tfidf.predict(X_test_tfidf)

print("Accuracy :", accuracy_score(y_test, y_pred_svm_tfidf))
print("Precision:", precision_score(y_test, y_pred_svm_tfidf, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_svm_tfidf, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_svm_tfidf, average="weighted"))
```

```
print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred_svm_tfidf))

print("\nClassification Report")
print(classification_report(y_test, y_pred_svm_tfidf))

import numpy as np
import pandas as pd

def show_all_model_prediction(text):

  models = {
    "Naive Bayes (BOW)": (nb, bow),
    "Logistic Regression (BOW)": (lr, bow),
    "SVM (BOW)": (svm, bow),
    "Naive Bayes (TF-IDF)": (nb_tfidf, tfidf),
    "Logistic Regression (TF-IDF)": (lr_tfidf, tfidf),
    "SVM (TF-IDF)": (svm_tfidf, tfidf)
  }

  print("=" * 70)
  print("Input Text:")
  print(text)
  print("=" * 70)

  for name, (model, vectorizer) in models.items():

    try:

      x = vectorizer.transform([text])

      prediction = model.predict(x)[0]

      if hasattr(model, "predict_proba"):

        probabilities = model.predict_proba(x)[0]

      else:

        scores = model.decision_function(x)

        if len(scores.shape) > 1:
          scores = scores[0]

        exp_scores = np.exp(scores - np.max(scores))
```

```
            probabilities = exp_scores / exp_scores.sum()

        classes = model.classes_

        result = pd.DataFrame({
            "Class": classes,
            "Confidence": probabilities * 100
        })

        result = result.sort_values(
            by="Confidence",
            ascending=False
        )

        print("\nMODEL:", name)
        print("Prediction:", prediction)

        print(
            result.to_string(
                index=False,
                formatters={
                    "Confidence": "{:.2f}%".format
                }
            )
        )

        print("-" * 70)

    except Exception as e:

        print("\nMODEL:", name)
        print("ERROR:", e)
        print("-" * 70)

show_all_model_prediction(
    """

can't do anything in life

"""
)
```

Dataset 1 final result :

Algorithm	Representation	Dataset 1 (Accuracy)
Multinomial Naïve Bayes	Bag of Words	**86.15%**
Multinomial Naïve Bayes	TF-IDF	**60.37%**
Logistic Regression	Bag of Words	**98.39%**
Logistic Regression	TF-IDF	**97.11%**
LinearSVC	Bag of Words	**98.21%**
LinearSVC	TF-IDF	**98.04%**

Dataset 2: Emotion Detection Dataset

The second dataset used in this project is an emotion detection dataset obtained from the `dair-ai/emotion` dataset available through Hugging Face. The dataset is based on the CARER dataset introduced by Saravia et al. (2018) and contains English text messages collected from Twitter. The objective of using this dataset is to classify each text into one of six basic emotion categories.

The dataset contains six emotion classes, represented by numerical labels as follows:

Label	**Emotion**
0	Sadness
1	Joy
2	Love
3	Anger
4	Fear
5	Surprise

The dataset provides predefined training and testing splits. The official training set contains **16,000 records**, while the official test set contains **2,000 records**. In this project, the official

train/test splits were used instead of creating a separate 80/20 split. This was done to maintain the standard benchmark configuration provided by the dataset.

The dataset contains short English text messages expressing different emotional states. Initial inspection of the dataset showed that the text was already largely normalized. The text was found to be lowercase and largely free from punctuation, hashtags, URLs, and user mentions. Therefore, separate preprocessing operations for removing punctuation, URLs, hashtags, and mentions were not necessary for this dataset.

The class distribution is not perfectly balanced. The Joy and Sadness categories contain the largest proportions of samples, while Love and Surprise are minority classes. This class imbalance is considered during model evaluation by using macro-average F1 and weighted-average F1 scores in addition to accuracy.

The official training and testing partitions were retained throughout the experiment. The feature vectorizers were fitted only using the training data and subsequently applied to the test data. This prevents information from the test set from influencing the feature extraction process and avoids data leakage.

```
from datasets import load_dataset


dataset = load_dataset("dair-ai/emotion")
df = dataset["train"].to_pandas()
test_df = dataset["test"].to_pandas()
df.head()
```

```
labelValue = {
    0: "sadness",
    1: "joy",
    2: "love",
    3: "anger",
    4: "fear",
    5: "surprise"
}
df["label"].value_counts()
```

	text	label
0	i didnt feel humiliated	0
1	i can go from feeling so hopeless to so damned...	0

2	im grabbing a minute to post i feel greedy wrong	3
3	i am ever feeling nostalgic about the fireplac...	2
4	i am feeling grouchy	3

label
1 5362
0 4666
3 2159
4 1937
2 1304
5 572
Name: count, dtype: int64

Project Implementation Detail

Task 1: Import Required Libraries

Description of the Task

The first step of this project is to import all the required Python libraries that are used throughout the implementation. Each library provides specific functionalities for data handling, natural language processing, feature extraction, machine learning, and performance evaluation.

The imported libraries prepare the programming environment for loading the emotion dataset, preprocessing the text, converting the text into numerical feature representations, training classification algorithms, and evaluating the prediction results.

For Dataset 2, spaCy is used for linguistic preprocessing and lemmatization, NLTK is used for stop-word processing, and Scikit-learn is used for vectorization, classification, and evaluation.

```
import pandas as pd
import numpy as np


import nltk
import string
import spacy
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
```

```
from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
import spacy

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
print('done')
```

Code Description

The code imports the libraries required for the Dataset 2 emotion classification task.

Pandas and NumPy are used for data handling and numerical operations. NLTK provides the English stop-word list used during preprocessing, while spaCy provides natural language processing functionality, including tokenization and lemmatization.

CountVectorizer and TfidfVectorizer are imported to convert the preprocessed text into numerical representations using Bag of Words and TF-IDF.

The three machine learning algorithms used in the experiment are Multinomial Naïve Bayes, Logistic Regression, and LinearSVC.

The Scikit-learn evaluation functions are used to measure classification performance using accuracy and classification reports. Confusion matrices are also generated to analyze the classification behavior of the models.

Stop-word and spacy for lemmatization was imported for future use.

Task 2: Dataset Loading

Description of the Task

In this task, the Emotion Dataset is downloaded from the Hugging Face dataset repository(https://huggingface.co/datasets/dair-ai/emotion) using the datasets library.

The official training and testing partitions provided by the dataset are loaded separately. The training partition contains **16,000 records**, while the testing partition contains **2,000 records**.

The training split is converted into a Pandas DataFrame named df, while the test split is converted into another DataFrame named test_df.

Unlike Dataset 1, a separate train-test split is not performed for Dataset 2 because the dataset already provides official predefined training and testing partitions. Using these predefined partitions maintains the standard dataset configuration.

```
from datasets import load_dataset


dataset = load_dataset("dair-ai/emotion")

df = dataset["train"].to_pandas()

test_df = dataset["test"].to_pandas()
```

```
README.md:

 9.05k/?  [00:00<00:00,  710kB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.


split/train-00000-of-00001.parquet:  100%

 1.03M/1.03M  [00:00<00:00,  5.14MB/s]
```

```
split/validation-00000-of-00001.parquet:  100%
 127k/127k  [00:00<00:00,  637kB/s]
split/test-00000-of-00001.parquet:  100%
 129k/129k  [00:00<00:00,  577kB/s]
Generating  train  split:  100%
 16000/16000  [00:00<00:00,  174980.94  examples/s]
Generating  validation  split:  100%
 2000/2000  [00:00<00:00,  118136.10  examples/s]
Generating  test  split:  100%
 2000/2000  [00:00<00:00,  124646.84  examples/s]
```

Code Description

The code loads the dair-ai/emotion dataset from Hugging Face.

The official training split is converted into the df DataFrame, while the official testing split is converted into test_df.

The two DataFrames are kept separate throughout the experiment. This ensures that the test data remains unseen during model training and feature-vectorizer fitting.

Task 3: Data Inspection

Description of the Task

After loading the dataset, the first five records of the training DataFrame are displayed to inspect the structure and content of the dataset.

The inspection allows the raw text and corresponding numerical emotion labels to be observed before preprocessing.

The labels are represented using integers from 0 to 5, where each integer corresponds to one of the six emotion categories.

df.head()

	text	label
0	i didnt feel humiliated	0
1	i can go from feeling so hopeless to so damned...	0
2	im grabbing a minute to post i feel greedy wrong	3
3	i am ever feeling nostalgic about the fireplac...	2
4	i am feeling grouchy	3

Task 4: Label Distribution

Description of the Task

In this task, the distribution of the emotion classes in the training dataset is examined.

A dictionary is created to map the numerical labels to their corresponding emotion names. The frequency of each numerical label is then calculated using value_counts().

This analysis is important because it identifies whether the dataset is balanced across all emotion categories.

The analysis shows that the dataset is imbalanced, with **Joy and Sadness** representing relatively larger classes, while **Surprise** is one of the minority classes.

```
labelValue = {
    0: "sadness",
    1: "joy",
    2: "love",
    3: "anger",
    4: "fear",
    5: "surprise"
}
df["label"].value_counts()
```

```
label
1    5362
0    4666
3    2159
4    1937
2    1304
```

```
5     572
Name: count, dtype: int64
```

Code Description

The label mapping dictionary associates each numerical label with its corresponding emotion name.

The value_counts() function counts the number of samples belonging to each emotion class.

The resulting distribution demonstrates that the six classes do not contain equal numbers of samples. Therefore, accuracy alone may not fully describe the performance of the models. Macro-average and weighted-average F1 scores are also considered during evaluation to provide a more complete understanding of model performance

Task 5: Missing Value Handling

Description of the Task

Before beginning the preprocessing stage, missing values are removed from both the training and testing DataFrames.

The dropna() operation is applied to remove rows containing missing values in any column.

Although the original Emotion Dataset does not contain missing values in the text or label fields, this step ensures that the preprocessing and machine learning pipeline does not encounter errors caused by unexpected null values.

```
df.dropna(how='any', inplace=True)

test_df.dropna(how='any', inplace=True)
```

Code Description

The dropna(how='any') operation removes any row containing a missing value.

The operation is applied separately to the training and testing DataFrames.

Since the dataset does not contain missing values requiring removal, the size of the DataFrames remains unchanged. Nevertheless, the operation provides an additional data-quality check before preprocessing.

Task 6: Lemmatization of Training Data

Description of the Task

In this task, lemmatization is applied to the text in the training dataset.

The pretrained spaCy English language model en_core_web_sm is loaded and used to identify the base or dictionary form of words.

Lemmatization is used to reduce variations of words into their meaningful base forms. This can reduce unnecessary vocabulary variation and allow the machine learning models to treat related word forms more consistently.

The lemmatized text is stored in a new column named lemmatized.

```
nlp = spacy.load("en_core_web_sm")

def lemmatize_text(text):
    doc = nlp(text)
    return " ".join([token.lemma_ for token in doc])


df["lemmatized"] = df["text"].apply(lemmatize_text)
```

	text	label	lemmatized	lemma_text	clean_text	
0	im feeling rather rotten so im not very ambiti...	0	I m feel rather rotten so I m not very ambitio...	i m feel rather rotten so i m not very ambitio...	feel rather rotten not ambitious right	
1	im updating my blog because i feel shitty	0	I m update my blog because I feel shitty	i m update my blog because i feel shitty	update blog feel shitty	
2	i never make her separate from me because i do...	0	I never make she separate from I because I don...	i never make she separate from i because i don...	never make separate ever want feel like ashamed	
3	i left with my bouquet of red and yellow tulip...	1	I leave with my bouquet of red and yellow tuli...	i leave with my bouquet of red and yellow tuli...	leave bouquet red yellow tulip arm feel slight...	
4	i was feeling a little vain when i did this one	0	I be feel a little vain when I do this one	i be feel a little vain when i do this one	feel little vain one	

Code Description

The code loads spaCy's pretrained English language model and defines a function for lemmatizing text.

The function processes the text and extracts the lemma of each token. The resulting words are then combined to form the lemmatized version of the original text.

The lemmatization process is applied to the training data and the resulting text is stored in the lemmatized column.

A pretrained spaCy model is required here because Dataset 2 uses actual lemmatization. A blank spaCy model, such as the one used for simple tokenization in Dataset 1, would not provide the linguistic information necessary for meaningful lemmatization.

Task 7: Lemmatization of Test Data

Description of the Task

The same lemmatization process is applied to the official test dataset.

The text in test_df["text"] is processed using the same spaCy lemmatization function used for the training data. The resulting lemmatized text is stored in the lemmatized column of the test DataFrame.

Using the same preprocessing procedure for both training and testing data ensures consistency between the data used to train the models and the unseen data used for evaluation.

```
nlp = spacy.load("en_core_web_sm")

def lemmatize_text(text):
    doc = nlp(text)
    return " ".join([token.lemma_ for token in doc])


test_df["lemmatized"] = test_df["text"].apply(lemmatize_text)
```

	text	label	lemmatized	lemma_text	clean_text	
0	im feeling rather rotten so im not very ambiti...	0	I m feel rather rotten so I m not very ambitio...	i m feel rather rotten so i m not very ambitio...	feel rather rotten not ambitious right
1	im updating my blog because i feel shitty	0	I m update my blog because I feel shitty	i m update my blog because i feel shitty	update blog feel shitty
2	i never make her separate from me because i do...	0	I never make she separate from I because I don...	i never make she separate from i because i don...	never make separate ever want feel like ashamed
3	i left with my bouquet of red and yellow tulip...	1	I leave with my bouquet of red and yellow tuli...	i leave with my bouquet of red and yellow tuli...	leave bouquet red yellow tulip arm feel slight...
4	i was feeling a little vain when i did this one	0	I be feel a little vain when I do this one	i be feel a little vain when i do this one	feel little vain one

Code Description

The lemmatization function defined previously is applied to every text sample in the official test dataset.

The resulting lemmatized text is stored in test_df["lemmatized"].

The same preprocessing procedure is applied independently to the training and test datasets without combining them. This maintains the separation between training and evaluation data.

Task 8: Lowercasing

Description of the Task

After lemmatization, the resulting text is converted into lowercase for both the training and testing datasets.

Although the original dataset is already largely lowercase, this step is necessary because the spaCy lemmatization process may reintroduce capitalization in certain cases. For example, the pronoun "I" may be returned as an uppercase lemma.

Therefore, lowercasing is applied after lemmatization to ensure consistent text representation before stop-word removal and vectorization.

The processed training text is stored as final_text, while the processed test text is stored as lemma_text.

```
df["final_text"] = df["lemmatized"].str.lower()

test_df["lemma_text"] = test_df["lemmatized"].str.lower()

df.head()
```

	text	label	lemmatized	final_text	
0	i didnt feel humiliated	0	I do not feel humiliated	i do not feel humiliated	
1	i can go from feeling so hopeless to so damned...	0	I can go from feel so hopeless to so damned ho...	i can go from feel so hopeless to so damned ho...	
2	im grabbing a minute to post i feel greedy wrong	3	I m grab a minute to post I feel greedy wrong	i m grab a minute to post i feel greedy wrong	
3	i am ever feeling nostalgic about the fireplac...	2	I be ever feel nostalgic about the fireplace I...	i be ever feel nostalgic about the fireplace i...	
4	i am feeling grouchy	3	I be feel grouchy	i be feel grouchy	

Code Description

The input variables contain the preprocessed text that will be used as features for the machine learning models.

The target variables contain the numerical emotion labels that the models must learn to predict.

The training data contains 16,000 samples, while the test data contains 2,000 samples.

No additional train_test_split() operation is performed because the dataset already provides official training and testing partitions.

Task 9: Customized Stop-Word Removal

Description of the Task

Stop-word removal is applied after lemmatization and lowercasing.

The default English stop-word list provided by NLTK is customized for the emotion classification task.

The words not, no, and nor are removed from the default stop-word list so that they are retained in the text. These words represent negation and may significantly change the emotional meaning of a sentence.

For example, removing the word "not" from a sentence such as "I am not happy" could cause the resulting text to lose an important part of its emotional meaning.

In addition, several contraction-related tokens are included in the removal set as noise or broken-contraction artifacts. These include tokens such as im, ive, youre, theyre, hes, shes, weve, youve, id, ill, and youll.

The customized stop-word removal process is applied consistently to both the training and testing datasets.

```
extra_noise_words = {"im", "ive", "youre", "theyre", "hes", "shes", "weve",
                     "youve", "id", "ill", "youll"}
stop_words = (set(stopwords.words('english')) - {"not", "no", "nor"}) |
extra_noise_words


def remove_stopwords(text):
    return " ".join([w for w in text.split() if w not in stop_words])


df["clean_text"] = df["final_text"].apply(remove_stopwords)
test_df["clean_text"] = test_df["lemma_text"].apply(remove_stopwords)
```

Code Description

The default NLTK English stop-word set is first modified according to the requirements of the emotion classification task.

The negation words not, no, and nor are excluded from removal to preserve important emotional and semantic information.

Additional contraction-related tokens are added to the stop-word set because they were considered low-value noise or broken-contraction artifacts for this dataset.

The customized stop-word set is then applied to the lemmatized and lowercased training and testing text.

Task 10: Splitting Predictors and Targets

Description of the Task

After completing the preprocessing stage, the input text and target labels are separated into predictor and target variables.

The processed training text is assigned to X_train, while the corresponding emotion labels are assigned to y_train.

Similarly, the processed test text is assigned to X_test, while the corresponding labels are assigned to y_test.

The official training and testing partitions remain separate throughout this process.

```
X_train = df["clean_text"]

y_train = df["label"]

X_test = test_df["clean_text"]

y_test = test_df["label"]
```

Code Description

The input variables contain the preprocessed text that will be used as features for the machine learning models.

The target variables contain the numerical emotion labels that the models must learn to predict.

The training data contains 16,000 samples, while the test data contains 2,000 samples.

No additional train_test_split() operation is performed because the dataset already provides official training and testing partitions.

Task 11: Text Vectorization Using BoW and TF-IDF

Description of the Task

Machine learning algorithms cannot directly process raw text. Therefore, the preprocessed text is converted into numerical feature representations.

Two vectorization techniques are used:

Bag of Words (BoW)

Term Frequency-Inverse Document Frequency (TF-IDF)

The Bag of Words representation is generated using CountVectorizer, while the TF-IDF representation is generated using TfidfVectorizer.

Both vectorizers are fitted strictly on the training text. The trained vectorizers are then used to transform the test text.

This approach is important for preventing data leakage. The test data is never used to fit the vectorizers, meaning that information from the test set does not influence the vocabulary or feature weights used during model training.

```
bow = CountVectorizer()
X_train_bow = bow.fit_transform(X_train)
X_test_bow = bow.transform(X_test)


tfidf = TfidfVectorizer()
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

Code Description

CountVectorizer converts the preprocessed text into a Bag of Words numerical representation based on word occurrence.

TfidfVectorizer creates a TF-IDF representation by assigning weights to words according to their importance within the documents.

Both vectorizers are fitted using only X_train. The resulting fitted vectorizers are then used to transform X_test.

The test data is therefore processed using .transform() rather than being fitted again. This prevents data leakage and ensures that the evaluation represents the performance of models on unseen data.

Task 12: Model Training and Evaluation

Description of the Task

In this task, three machine learning classification algorithms are trained and evaluated using both Bag of Words and TF-IDF representations.

The algorithms used are:

- Multinomial Naïve Bayes
- Logistic Regression
- LinearSVC

Each algorithm is tested using both feature representations, resulting in six experimental configurations.

- The configurations are:
- Naïve Bayes + BoW
- Naïve Bayes + TF-IDF
- Logistic Regression + BoW
- Logistic Regression + TF-IDF
- LinearSVC + BoW
- LinearSVC + TF-IDF

The trained models are evaluated using classification reports containing precision, recall, F1 score, macro-average F1, and weighted-average F1.

Logistic Regression uses max_iter=1000 to provide sufficient iterations for model convergence.

The zero_division=0 parameter is used during classification report generation to prevent undefined-metric warnings in cases where a model does not predict samples belonging to a particular class.

```
nb_bow = MultinomialNB().fit(X_train_bow, y_train)

y_pred = nb_bow.predict(X_test_bow)

print("===== Naive Bayes - BoW =====")

print(classification_report(y_test, y_pred))



nb_tfidf = MultinomialNB().fit(X_train_tfidf, y_train)

y_pred = nb_tfidf.predict(X_test_tfidf)
```

```
print("===== Naive Bayes - TF-IDF =====")
print(classification_report(y_test, y_pred))



lr_bow = LogisticRegression(max_iter=1000).fit(X_train_bow, y_train)
y_pred = lr_bow.predict(X_test_bow)
print("===== Logistic Regression - BoW =====")
print(classification_report(y_test, y_pred))



lr_tfidf = LogisticRegression(max_iter=1000).fit(X_train_tfidf, y_train)
y_pred = lr_tfidf.predict(X_test_tfidf)
print("===== Logistic Regression - TF-IDF =====")
print(classification_report(y_test, y_pred))



svm_bow = LinearSVC().fit(X_train_bow, y_train)
y_pred = svm_bow.predict(X_test_bow)
print("===== SVM - BoW =====")
print(classification_report(y_test, y_pred))



svm_tfidf = LinearSVC().fit(X_train_tfidf, y_train)
y_pred = svm_tfidf.predict(X_test_tfidf)
print("===== SVM - TF-IDF =====")
print(classification_report(y_test, y_pred))
```

```
===== Naive Bayes - BoW =====
              precision    recall  f1-score   support

           0       0.75      0.92      0.83       581
           1       0.77      0.96      0.85       695
           2       0.90      0.28      0.43       159
           3       0.90      0.64      0.75       275
           4       0.84      0.63      0.72       224
```

```
           5       1.00      0.05      0.09        66

    accuracy                           0.79      2000
   macro avg       0.86      0.58      0.61      2000
weighted avg       0.81      0.79      0.76      2000

===== Naive Bayes - TF-IDF =====
              precision    recall  f1-score   support

           0       0.69      0.91      0.78       581
           1       0.66      0.99      0.79       695
           2       1.00      0.07      0.13       159
           3       0.95      0.33      0.49       275
           4       0.88      0.34      0.49       224
           5       0.00      0.00      0.00        66

    accuracy                           0.70      2000
   macro avg       0.70      0.44      0.45      2000
weighted avg       0.74      0.70      0.64      2000

===== Logistic Regression - BoW =====
              precision    recall  f1-score   support

           0       0.91      0.92      0.91       581
           1       0.88      0.92      0.90       695
           2       0.75      0.67      0.71       159
           3       0.86      0.85      0.86       275
           4       0.87      0.86      0.87       224
           5       0.78      0.59      0.67        66

    accuracy                           0.87      2000
   macro avg       0.84      0.80      0.82      2000
weighted avg       0.87      0.87      0.87      2000

===== Logistic Regression - TF-IDF =====
              precision    recall  f1-score   support

           0       0.88      0.92      0.90       581
           1       0.83      0.95      0.88       695
           2       0.80      0.55      0.65       159
           3       0.89      0.78      0.83       275
           4       0.89      0.81      0.85       224
           5       0.88      0.44      0.59        66

    accuracy                           0.85      2000
   macro avg       0.86      0.74      0.78      2000
weighted avg       0.86      0.85      0.85      2000

===== SVM - BoW =====
              precision    recall  f1-score   support
```

```
              0       0.91      0.90      0.91       581
              1       0.89      0.90      0.90       695
              2       0.73      0.74      0.73       159
              3       0.86      0.86      0.86       275
              4       0.84      0.86      0.85       224
              5       0.73      0.67      0.70        66

    accuracy                           0.87      2000
   macro avg       0.83      0.82      0.82      2000
weighted avg       0.87      0.87      0.87      2000

===== SVM - TF-IDF =====
              precision    recall  f1-score   support

              0       0.91      0.92      0.92       581
              1       0.89      0.93      0.91       695
              2       0.77      0.70      0.73       159
              3       0.86      0.85      0.86       275
              4       0.88      0.84      0.86       224
              5       0.70      0.64      0.67        66

    accuracy                           0.88      2000
   macro avg       0.83      0.81      0.82      2000
weighted avg       0.87      0.88      0.87      2000
```

Code Description

The code trains the three selected classification algorithms using both Bag of Words and TF-IDF feature representations.

Multinomial Naïve Bayes is trained separately using BoW and TF-IDF features. Logistic Regression is similarly trained using both representations, followed by LinearSVC.

The trained models make predictions on the unseen test data. Their predictions are then evaluated using classification reports.

This process allows the effect of both the machine learning algorithm and the vectorization technique to be compared under the same preprocessing conditions.

Task 13: Confusion Matrix Visualization

Description of the Task

In the final stage of the implementation, a confusion matrix is generated to visualize the classification performance of the final evaluated model.

The six emotion labels are explicitly defined as:

- Sadness

- Joy
- Love
- Anger
- Fear
- Surprise

The confusion matrix compares the actual emotion labels with the predicted labels.

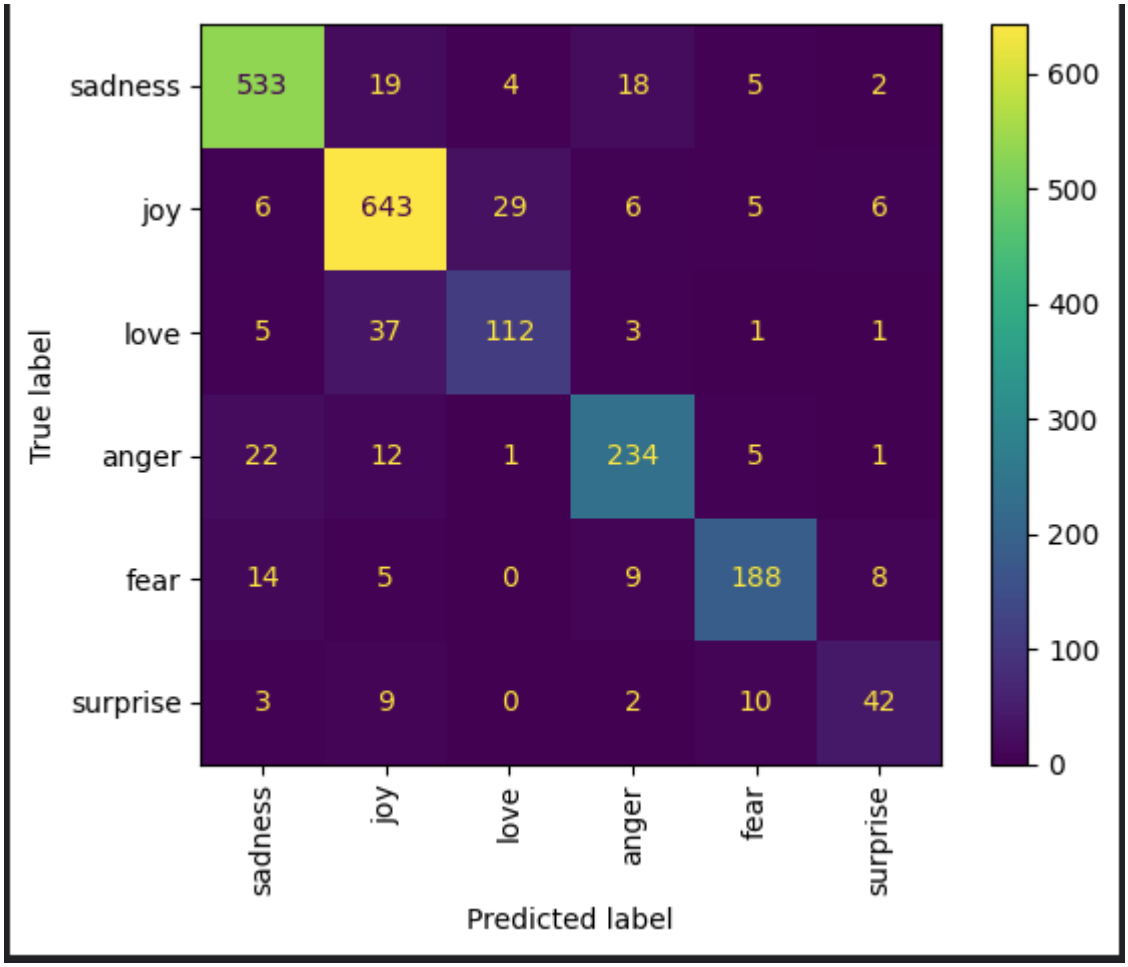
The diagonal elements represent correctly classified samples, while the off-diagonal elements represent samples that were incorrectly classified as another emotion.

The confusion matrix provides a visual representation of the classification behavior of the model and can help identify which emotion categories are more frequently confused with one another.

In the current implementation, the confusion matrix is generated for the LinearSVC model using TF-IDF, which is the final model evaluated in the previous cell.

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay


emotion_labels = ["sadness", "joy", "love", "anger", "fear", "surprise"]
ConfusionMatrixDisplay.from_predictions(
   y_test,
   y_pred,
   display_labels=emotion_labels,
   xticks_rotation="vertical"
)
```



Code Description

The code defines the names of the six emotion categories and uses ConfusionMatrixDisplay.from_predictions() to generate a confusion matrix.

The matrix displays the relationship between the actual emotion labels and the labels predicted by the LinearSVC model trained using TF-IDF features.

The diagonal cells indicate correct predictions, while the non-diagonal cells indicate misclassifications.

The confusion matrix can be used alongside the classification report to identify the specific classes that the model handles well and the classes that are more frequently confused.

Final Whole Code:

```
import pandas as pd

import numpy as np
```

```
import nltk
import string
import spacy
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
print('done')
```

```
from datasets import load_dataset


dataset = load_dataset("dair-ai/emotion")
```

```
df = dataset["train"].to_pandas()
test_df = dataset["test"].to_pandas()
```

```
df.head()
```

```
labelValue = {
   0: "sadness",
   1: "joy",
   2: "love",
   3: "anger",
   4: "fear",
   5: "surprise"
}
df["label"].value_counts()
```

```
df.dropna(how='any', inplace=True)
test_df.dropna(how='any', inplace=True)
```

```
nlp = spacy.load("en_core_web_sm")
def lemmatize_text(text):
   doc = nlp(text)
   return " ".join([token.lemma_ for token in doc])

df["lemmatized"] = df["text"].apply(lemmatize_text)
```

```
nlp = spacy.load("en_core_web_sm")
def lemmatize_text(text):
   doc = nlp(text)
   return " ".join([token.lemma_ for token in doc])

test_df["lemmatized"] = test_df["text"].apply(lemmatize_text)
```

```
test_df.head()
```

```
df["final_text"] = df["lemmatized"].str.lower()
test_df["lemma_text"] = test_df["lemmatized"].str.lower()
df.head()
```

```
extra_noise_words = {"im", "ive", "youre", "theyre", "hes", "shes", "weve",
                     "youve", "id", "ill", "youll"}
stop_words = (set(stopwords.words('english')) - {"not", "no", "nor"}) |
extra_noise_words


def remove_stopwords(text):
    return " ".join([w for w in text.split() if w not in stop_words])


df["clean_text"] = df["final_text"].apply(remove_stopwords)
test_df["clean_text"] = test_df["lemma_text"].apply(remove_stopwords)
```

```
X_train = df["clean_text"]
y_train = df["label"]
X_test = test_df["clean_text"]
y_test = test_df["label"]
```

```
bow = CountVectorizer()
X_train_bow = bow.fit_transform(X_train)
X_test_bow = bow.transform(X_test)


tfidf = TfidfVectorizer()
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

```
nb_bow = MultinomialNB().fit(X_train_bow, y_train)
y_pred = nb_bow.predict(X_test_bow)
print("===== Naive Bayes - BoW =====")
print(classification_report(y_test, y_pred))
```

```
nb_tfidf = MultinomialNB().fit(X_train_tfidf, y_train)
y_pred = nb_tfidf.predict(X_test_tfidf)
print("===== Naive Bayes - TF-IDF =====")
print(classification_report(y_test, y_pred))


lr_bow = LogisticRegression(max_iter=1000).fit(X_train_bow, y_train)
y_pred = lr_bow.predict(X_test_bow)
print("===== Logistic Regression - BoW =====")
print(classification_report(y_test, y_pred))


lr_tfidf = LogisticRegression(max_iter=1000).fit(X_train_tfidf, y_train)
y_pred = lr_tfidf.predict(X_test_tfidf)
print("===== Logistic Regression - TF-IDF =====")
print(classification_report(y_test, y_pred))


svm_bow = LinearSVC().fit(X_train_bow, y_train)
y_pred = svm_bow.predict(X_test_bow)
print("===== SVM - BoW =====")
print(classification_report(y_test, y_pred))


svm_tfidf = LinearSVC().fit(X_train_tfidf, y_train)
y_pred = svm_tfidf.predict(X_test_tfidf)
print("===== SVM - TF-IDF =====")
print(classification_report(y_test, y_pred))
```

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay


emotion_labels = ["sadness", "joy", "love", "anger", "fear", "surprise"]
ConfusionMatrixDisplay.from_predictions(
   y_test,
   y_pred,
   display_labels=emotion_labels,
   xticks_rotation="vertical"
)
```

Results for Dataset 2

The performance of the three machine learning algorithms was evaluated using both Bag of Words and TF-IDF feature representations. The results obtained from the official test set containing 2,000 samples are presented below.

Algorithm	Representation	Accuracy	Macro Avg F1	Weighted Avg F1
NB	BoW	0.79	0.61	0.76
NB	TF-IDF	0.70	0.45	0.64
LR	BoW	0.87	0.82	0.87
LR	TF-IDF	0.85	0.78	0.85
SVM	BoW	0.87	0.82	0.87
SVM	TF-IDF	**0.88**	**0.82**	**0.87**

Results Analysis

The results demonstrate that the choice of machine learning algorithm and feature representation significantly affects emotion classification performance.

Multinomial Naïve Bayes achieved an accuracy of **0.79** with Bag of Words. However, its performance decreased to **0.70** when TF-IDF was used. The macro-average F1 score also decreased from **0.61** to **0.45**, indicating weaker performance across the six emotion classes.

The most significant problem occurred with the **Surprise** class when Naïve Bayes was combined with TF-IDF. The model failed to predict any samples as Surprise, resulting in a recall and F1 score of **0.00** for that class. This demonstrates that the combination of Multinomial Naïve Bayes and TF-IDF was particularly ineffective for the minority class in this dataset.

Logistic Regression performed substantially better than Naïve Bayes. Using Bag of Words, it achieved an accuracy of **0.87**, macro-average F1 of **0.82**, and weighted-average F1 of **0.87**. With TF-IDF, it achieved an accuracy of **0.85**, macro-average F1 of **0.78**, and weighted-average F1 of **0.85**.

LinearSVC also achieved strong performance using both representations. With Bag of Words, it achieved **0.87 accuracy**, **0.82 macro-average F1**, and **0.87 weighted-average F1**. Its performance improved slightly when TF-IDF was used, reaching the highest accuracy of **0.88**.

Therefore, the **LinearSVC with TF-IDF** configuration achieved the highest overall accuracy among the six tested configurations.

The difference between macro-average and weighted-average F1 scores also highlights the effect of class imbalance. Weighted-average F1 is influenced more heavily by the larger emotion classes, whereas macro-average F1 gives equal importance to every class. The lower macro-average scores in some configurations indicate that the models do not perform equally well across all six emotion categories.

Overall, LinearSVC with TF-IDF provided the strongest overall performance for Dataset 2, while Multinomial Naïve Bayes showed the weakest performance, particularly when combined with TF-IDF.

Structure followed for this Project:

Imports → Dataset Loading → Inspection → Label Distribution → Missing Values → Train Lemmatization → Test Lemmatization → Lowercasing → Stopword Removal → X/y Separation → BoW & TF-IDF → Model Training & Evaluation → Confusion Matrix